

1 Team Info

1.1 과제명

청소가 어려운 자취 초년생을 위한 청소 어시스턴트, AI 쓱싹비서

1.2 팀정보

25팀 바위너구리

1.3 팀 구성원 및 역할

2271051 이해인 : 팀장 / **AI** 개발

- 청소 스케줄링 시스템의 핵심 알고리즘 개발 주도
- ILP 및 강화학습 기반 최적화 로직을 설계·구현
- 청소 일정 자동화 서비스 플로우를 고려해 ILP와 RL의 결합 구조 설계
- PuLP를 활용한 ILP 기반 청소 일정 초안 생성 로직 구현
- Tianshou(PPO)를 적용한 강화학습 업그레이드 로직 구현
- 사용자 일정·수면 데이터·청소 이력 통합 처리 및 모델 연동
- 청소 일정의 합리성과 RL 학습 결과를 다양한 시뮬레이션으로 검증
- 모델 출력과 사용자 요구사항이 일치하는지 확인하고 개선점을 반영

2122004 고은서 : 팀원 / **FE** 개발

- 프로젝트 FE 총괄 담당
- 사용자 요구에 맞는 UI/UX 흐름 설계(와이어프레임, 화면 구조도 작성)
- FE-서버 통신 방식(API 명세, 데이터 포맷 협의) 설계
- 사용자 경험(UX) 검증 포인트 제안
- FE 페이지 개발 및 주요 화면 기능 구현
- FE-BE/AI 서버 연동 및 디버깅
- 실사용자 테스트 기반 UI/UX 개선 및 반응형 대응
- 최종 발표용 FE 시연 준비 및 안정화

2076458 황혜린 : 팀원 / **BE** 개발

- 백엔드 아키텍처, API 설계 주도
- 서비스 플로우 고려한 시스템 아키텍처 설계
- 보안, 확장성, 성능 등을 고려한 기술적 의사결정
- RESTful API 및 비즈니스 로직 구현, 데이터베이스 최적화 등 핵심 서버 로직 개발
- 요구사항 변화를 빠르게 반영하여 애자일하게 기능 개선
- 단위/통합 테스트를 주도 및 QA 과정에서 발생한 버그를 빠르게 수정
- 개발 의도, 기획 의도 일치 위한 검증 단계 브리징 역할 수행

2 Project Summary

2.1 문제 정의

최근 1인 가구 증가와 함께 주거 관리 및 위생 유지의 중요성이 강조되고 있으나 실제로는 청소 실행과 습관 형성에서 많은 사람들이 어려움을 겪고 있다. 내외경제TV가 보도한 1인 가구 청소 패턴 조사 결과¹에 따르면, “해야 할 집안일을 미루지 않고 바로 하는 편이다”는 응답은 전체의 17.7%에 불과하였으며 다수의 응답자는 청소를 지연하거나 몰아서 처리하는 경향을 보였다. 또한 청소도구 보급률은 69.9%로 높음에도 불구하고 실제 실행으로 이어지지 않는 점에서 문제의 본질이 도구의 부족이 아닌 실행 동기와 생활 패턴의 불규칙성임을 확인할 수 있다.

자체 설문조사(자체 설문조사, 2025.09.16.~09.20., 20대 1인 가구 48명 대상)에서도 유사한 결과가 나타났다. 응답자의 다수는 청소 일정을 스스로 계획하고 실행하는 데 어려움을 겪는다고 응답하였으며, 그 원인으로 시간 부족, 동기부여 결여, 몰아서 하는 습관 등을 지적하였다. 특히 단순한 필요성 인식에 그치지 않고, 응답자 중 상당수가 청소를 장기적 생활 습관으로 정착시키고자 하는 욕구를 드러냈다는 점에서 기존의 문제를 해결할 수 있는 새로운 지원 체계가 필요하다.

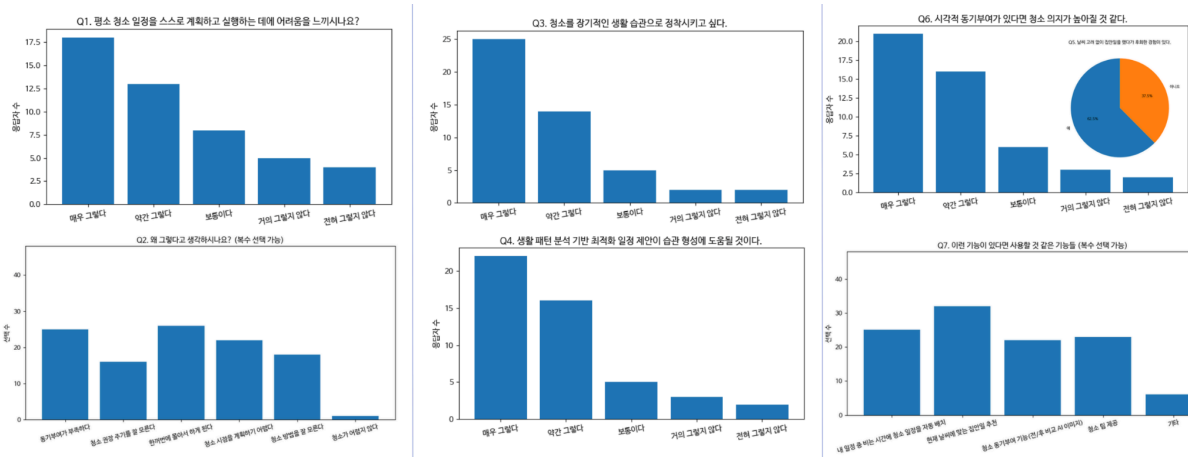


그림 1. 자체 설문조사 결과: 청소 일정 계획 및 실행 어려움과 습관화 욕구

따라서 본 프로젝트는 1인 가구 청년층이 지속적으로 청소를 실천하고 습관화할 수 있도록 지원하는 지능형 시스템의 부재를 문제로 정의한다. 이에 따라 개인의 일정, 생활 패턴, 주거 환경, 외부 요인(예: 날씨) 등을 종합적으로 고려하여 실행 가능성과 지속 가능성을 동시에 확보하는 청소 루틴 생성 및 동기부여 체계가 필요하다.

¹ 내외경제TV, “1인가구 청소 습관 조사 결과,” 내외경제TV, 2023.9.13, <https://www.nbntv.co.kr/news/articleView.html?idxno=967894>

2.2 기존 연구와의 비교

기존 연구 및 상용 서비스는 크게 두 가지 갈래로 나눌 수 있다.

첫째, 스마트홈 기기(예: 로봇청소기)는 물리적 청소의 부담을 경감시킨다는 점에서 의의가 있으나 금전적 부담, 그리고 사용자의 개인적 생활 패턴 반영이나 청소 습관 형성 유도에는 한계가 존재한다. 이러한 기기는 청소의 자동화라는 단편적 문제 해결에 초점을 두고 있어 생활 습관 정착이라는 근본적 과제를 해결하지 못한다.

둘째, 일정 관리 어플리케이션(예: 구글 캘린더, 일반 To-do 앱)은 일정 기록과 리마인드 기능에서 효과적이지만, 청소와 같은 생활습관형 일정 최적화에는 특화되어 있지 않다. 그리고 이러한 시스템은 개개인의 생활 데이터나 외부 환경 변수를 반영하지 못하기 때문에 사용자가 실제 실행에 옮기도록 유도하는 효과는 제한적이다.

이에 비해 본 프로젝트는 기존 접근의 한계를 극복하고, 다음과 같은 차별성을 제시한다.

1. ILP(정수선형계획, **Integer Linear Programming**) 기반 맞춤형 초기 스케줄 생성
 - 사용자의 일정, 수면 패턴, 주거 환경 데이터 등을 통합 분석하여, 고정 일정을 회피하면서도 실현 가능한 청소 루틴을 자동적으로 생성한다.
2. 강화학습 기반 적응형 개선
 - 사용자의 청소 완료율과 피드백 데이터를 보상 함수에 반영함으로써, 시간이 지날수록 더욱 개인화된 루틴을 제공한다.
3. 외부 환경 데이터 반영
 - 기상 데이터(예: 강수량, 미세먼지, 습도)를 실시간으로 수집 및 분석하여 반복적 일정이 아닌, 상황 적응적 청소 권장 항목을 제안한다.
4. 시각적 동기부여 메커니즘
 - 최신 시각 인공지능 기법(**Segment Anything Model, Grounded-SAM** 등)들을 활용하여 청소 후 공간의 가상 이미지를 생성 및 제공함으로써 사용자가 성취감을 직관적으로 경험하고 동기부여가 되도록 돕는다.

즉, 기존 연구가 물리적 자동화 또는 단순한 일정 관리 및 알림 기능에 머물렀다면, 본 프로젝트는 AI 최적화 기법(ILP, 강화학습)과 외부 데이터 연동, 시각적 피드백을 결합함으로써 1인 가구 청년층의 청소 습관 정착이라는 실제적 문제를 해결하는 데 의의를 지닌다.

2.3 제안 내용

본 제안은 자취 초년생이 청소를 이루는 생활 패턴을 개선하고 장기적인 습관 정착을 돕는 것을 목표로 한다. 이를 위해 서비스는 세 가지 기능을 제공한다. 첫째, 생활 패턴·주거 환경·일정·선호도를 반영해 개인 맞춤형 청소 루틴을 생성하고, 실행 기록과 피드백을 기반으로 강화학습을 통해 점차 개인화한다. 둘째, 기상 데이터를 반영해 강수·미세먼지·습도 등 환경 조건에 맞는 청소 권고안을 제시한다. 셋째, 사용자가 방 사진을 업로드하면 AI가 청소 후 모습을 생성해 시각적 성취감과 동기를 제공한다. 본 서비스는 이와 같이 데이터 기반 일정 추천과 환경·심리적 요인을 결합하여 단순 알림을 넘어 지속적 적응과 강화 학습을 수행하는 스마트 청소 코치로써 사용자의 청소 습관 정착을 돕는다.

2.3.1 개인 맞춤형 청소 루틴 생성

2.3.1.1 초기 스케줄 자동 생성

사용자의 생활 패턴, 주거 환경, 일정, 청소 선호도를 기반으로 실행 가능한 초기 청소 루틴을 산출한다.

- 생활 패턴 데이터: 평균 기상·취침 시간, 요일별 생활 리듬(사용자 입력)
- 주거 환경 데이터: 거주 면적, 방·화장실 개수(사용자 입력)
- 일정 데이터: 구글 캘린더 연동을 통한 주요 고정 일정 수집
- 청소 선호도 데이터: 선호 요일·시간대, 대청소 여부 등(사용자 입력)

수집된 데이터는 정수 선형계획법(ILP, **Pulp** 활용)으로 처리되어, 사용자의 고정 일정을 피해도 실현 가능한 청소 스케줄 초안을 생성한다.

2.3.1.2 루틴 적응형 개선

생성된 루틴은 고정된 일정이 아니라, 사용자의 실행 데이터와 피드백을 반영하여 점진적으로 개선되는 적응형 루틴이다.

- 실행 데이터: 루틴별 완료 여부, 지연 여부, 반복 미이행 항목
- 사용자 피드백: 주간 난이도 평가(쉬움·적정·과부하), 불필요 항목 의견, 신규 요구사항

본 데이터는 강화학습 알고리즘으로 처리된다.

- 상태(State): 일정, 생활 패턴, 루틴 이행을, 날씨
- 행동(Action): 청소 항목 배치·시간대 변경·주기 조정
- 보상(Reward): 루틴 완료율, 긍정 피드백, 장기 지속률

이를 통해 루틴은 사용 횟수가 늘어날수록 점점 더 사용자 친화적으로 진화하며, 단순한 일정 추천을 넘어 쓰면 쓸수록 똑똑해지는 청소 코치로 기능한다.

2.3.2 날씨 맞춤 청소 추천

사용자의 거주지 기상 데이터를 반영하여 환경 요인 최적화 청소 권고안을 제공한다.

- 데이터 수집: 사용자 거주지 입력 + 외부 기상 API(OpenWeatherMap, 기상청 등)
- 예시 규칙:
 - 48시간 내 강수 예보 → 빨래 지양, 실내 정리 집중
 - 미세먼지 농도 양호($80\mu\text{g}/\text{m}^3$ 이하) → 최소 30분 환기 권장
 - 습도·온도 이상치 → 곰팡이 방지용 청소 추천

이와 같은 기상 요소는 단순 알림에 그치지 않고, 루틴 스케줄링 강화학습 과정에 반영되어 현실성과 실행 가능성을 높인다.

2.3.3 시각적 동기부여 기능

사용자가 청소 전 방 사진을 업로드하면, AI 이미지 생성 모델(예: Stable Diffusion 기반 Fine-tuning)을 활용하여 청소 완료 후의 예상 모습을 시각적으로 제시한다.

- 입력: 방 사진 업로드 → 전처리(객체 감지, 레이아웃 분석)
- 출력: “정돈된 상태”의 방 이미지 생성
- 효과: 청소 실행 전후 대비 효과 제공 → 즉각적 성취감과 반복 실행 동기 강화

본 기능은 단순 스케줄링 이상의 심리적 강화 요인으로 작동하여, 장기적인 청소 습관 형성에 기여한다.

2.4 기대 효과 및 의의

본 프로젝트는 자취 초년생의 청소 습관 형성을 돕는 맞춤형 서비스 개발을 통해, 생활 편의성과 지속 가능한 자기 관리 습관 확립에 기여한다. 서비스는 개인화된 일정 추천, 환경 데이터 반영, 시각적 동기부여를 결합하여 단순 도구 제공을 넘어선 스마트 청소 코치로 기능한다.

2.4.1 생활 습관 개선 효과

- 청소 미루기 습관 완화: 자취 초년생들이 흔히 보이는 ‘몰아서 청소’ 패턴을 완화한다. 개인 맞춤형 루틴이 자동으로 제안되고, 미루기 패턴이 반복적으로 감지될 경우 루틴이 재조정되므로, 청소를 큰 부담이 아닌 작은 단위의 반복적 활동으로 경험하게 된다. 이를 통해 꾸준한 관리 습관 형성이 가능하다.
- 시간 관리 효율화: 사용자의 생활 패턴과 고정 일정을 기반으로 루틴이 최적화되므로, 공부·업무와 청소가 충돌하는 상황이 최소화된다. 청소 시간을 따로 고민하지 않고도 생활 리듬 속에서 자연스럽게 청소가 포함되며, 불필요한 스트레스와 시간 낭비를 줄인다.

- **심리적 안정성 확보:** 청소 실행 여부가 루틴 기록과 시각적 피드백으로 즉각 확인되므로, 사용자는 정돈된 환경에서 성취감과 자기 효능감을 느낄 수 있다. 이는 자취 초년생들이 독립 생활 속에서 흔히 겪는 무기력감·우울감 완화에도 긍정적으로 작용한다.
- **지속 가능한 자기 관리 습관 정착**
단기적인 이벤트성 관리가 아닌, 루틴의 적응형 개선 기능을 통해 장기적으로 유지 가능한 습관으로 발전한다. 사용자는 시간이 지남에 따라 점차 자신에게 최적화된 루틴을 경험하고, “꾸준히 관리하는 자기 주도적 생활 패턴”을 형성하게 된다.

2.4.2 기술적 의의

- **개인화 알고리즘 실증**
정수 선형계획법(ILP)과 강화학습을 결합한 루틴 최적화 기법을 생활 서비스 영역에 적용한다. 이를 통해 단순 알람이나 캘린더 알람 수준을 넘어, 사용자의 일정·습관·환경에 동적으로 적응하는 지능형 청소 일정 추천을 실증한다.
- **데이터 융합 활용**
일정 데이터, 생활 패턴, 주거 특성, 기상 정보 등 서로 다른 성격의 데이터를 통합하여 실행 가능하고 현실성 있는 청소 루틴을 제시한다. 이는 이질적 데이터를 단일 의사결정 체계로 통합하는 사례로서 기술적 의의가 크다.
- **맞춤형 AI 모델 개발**
본 프로젝트는 두 축에서 자체 AI 모델을 활용한다.
 - 루틴 최적화 보조 모델: 사용자의 실행 패턴과 피드백을 학습하여 강화학습과 결합, 시간이 지날수록 더 정밀한 일정 추천을 제공한다.
 - 시각적 피드백 모델: 범용 이미지 생성 모델과 달리, 실제 방 구조·가구 배치·청소 전후 데이터를 반영한 맞춤형 모델을 구축해 현실성 높은 청소 후 이미지를 제공한다. 이를 통해 청소 일정과 동기부여라는 두 영역에서 AI 기반 개인화 서비스의 가능성을 입증한다.
- **데이터 보안 및 독립성 확보**
자체 모델 운영을 통해 사용자 개인 데이터(사진·일정 등)가 외부 플랫폼에 의존하지 않고 서비스 내부에서 안전하게 처리된다. 이는 개인정보 보호와 서비스 신뢰성 측면에서 중요한 기술적 의의를 가진다.
- **모델 경량화 및 최적화**
범용 대규모 모델 대비 경량화된 구조를 채택하여 모바일·웹 환경에서 빠른 응답 속도를 제공한다. 이는 청소 루틴 추천과 시각적 피드백이 실시간 동기부여 기능으로 작동하는 데 필수적이다.

2.4.3 사회적 의의

- **청년 1인 가구의 생활 적응 지원:** 자취를 시작한 초년생들이 가장 크게 겪는 어려움 중 하나인 주거 관리 문제를 해결함으로써, 독립 초기의 생활 스트레스를 완화하고 안정적인 자기 관리 기반을 제공한다.
- **위생 및 정신 건강 증진:** 꾸준한 청소 습관 형성을 통해 쾌적한 생활 환경을 유지하게 함으로써, 청년층의 위생 수준과 정신적 안정감을 동시에 높인다. 이는 고립감·우울감 해소 등 정서적 측면에서도 긍정적 효과를 기대할 수 있다.
- **건강한 주거 문화 확산:** ‘청소를 미루는 습관’이 아닌 ‘꾸준히 관리하는 습관’을 사회적으로 확산시킴으로써, 청년층을 넘어 전 세대에 걸쳐 주거 위생 관리에 대한 인식 개선을 유도한다.

2.5 주요 기능 리스트

(1) ILP 기반 청소 스케줄 초안 생성

- **Solution**

- PuLP로 제약조건(총 청소 개수, 총 청소 시간, 고정 일정 제외, 동일 시간대 duration 합 ≤ 1 시간, 각 작업의 주간 최대 배정 횟수)을 설정해 목적 함수(시급한 청소 작업 최대 배정)를 최대화하는 스케줄을 산출한다.
- 과거 청소 데이터가 부족한 초기에는 랜덤성을 최소화해 강화학습 초기 수렴 속도를 높이는 데 사용한다.
- 실험
 - interval, last_done_days_ago 등을 활용해 우선순위가 높은 작업이 실제로 더 많이 배정되는지 확인.
 - 사용자 캘린더 일정·청소 달성률 등을 바꾸며 ILP가 유연하게 작동하는지 테스트.
- 결과
 - (2) 강화학습 기반 맞춤형 스케줄 고도화 ‘결과’에 함께 표시됨

(2) 강화학습 기반 맞춤형 스케줄 고도화

- **Solution**
 - Tianshou의 PPO 알고리즘을 이용해 사용자 행동 벡터(user_behavior), 지난주 피드백, 온보딩 데이터를 보상 함수에 반영한다.
 - 가입 시 수집하는 청소 선호 요일 및 시간대를 초기 데이터로 활용.
 - 강화학습 초기 랜덤성을 줄여 빠른 수렴 유도.
 - 보상 설계 항목:
 - 시간 슬롯 중복 선택 시 패널티


```
if is_duplicate:
    slot_penalty -= 2.0
```
 - 야간/새벽 시간대 선택 시 패널티


```
if start < 7 or start > 23.75:
    burden_penalty -= 0.5
```
 - 급한 청소 선택 시 보상: 많이 밀린 청소와 자주 해야하는 청소를 선택하면 점수가 높아지게 식을 세웠다.


```
frequency_score += (last_done_days_ago - interval + (7.0
if interval <= 4 else 0))
```
 - 7(일)x24(시간)칸의 시간 슬롯 당 청소 확률이 과거 청소 기록 기반으로 계산되어 behavior_vector에 저장되어있다.
 선택한 시간 슬롯에 청소 확률을 곱하여 추가할 보상을 계산한다.


```
preference_score += float(self.behavior_vector[idx]) * 1.5
```

 1.5는 0~1사이 확률값을 보정하기 위한 것이다.
 - 온보딩 데이터 활용
 - i) 사용자가 입력한 청소 선호 요일 및 시간대에 청소 태스크를 배치했으면 보상
 - ii) 사용자의 청소 스타일(한번에 몰아서/조금씩 나눠서)에 따라 청소 태스크들의 분포를 조정
- 실험
 - 10주간 시뮬레이션 코드(WEEKS=10) 결과 분석 후, 보상함수 가중치 및 수식을 수정하여 최적 성능 탐색.
 - fixed_schedules, behavior_vectors, weekly_todos를 기록하며 RL 전·후의 스케줄 적합도를 비교.
- 결과 : 사용자 온보딩 데이터 선호 요일(월, 수, 금) 반영됨을 시각적으로 확인 가능

	초안	최종 일정
--	----	-------

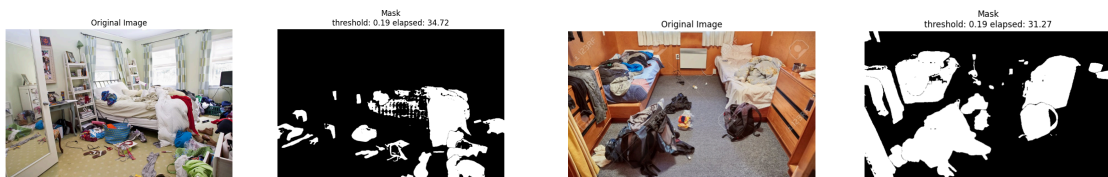
1주

1주차는 초안과 최종일정이 같습니다.

2주

10주

- **Grounded-Segment-Anything**으로 객체 마스킹, **Smart-Eraser**로 불필요한 물체 제거 및 배경 채움.
- 전·후 사진 비교·가상 청소 결과 제공으로 사용자의 청소 동기 강화.
- 실험
 - 다양한 방 이미지를 대상으로 마스킹 품질 테스트.
 - 마스킹 프롬프트 [**"trash"**, **"clothes"**, **"garbage bags"**, **"small objects"**]
 - 쓰레기와 정리가 필요한 물건을 흰색으로 탐지
- 결과
 - 목표 소요시간 달성: **30초**에 근접
 - 쓰레기와 정리 필요한 영역 탐지: 바닥의 짐, 침대 위 이불, 아무데나 놓인 옷
 - 한계: 아주 작거나 얇은 물체는 탐지하지 못 함



3

Project-Design

3.1 요구사항 정의

설계 모듈 및 모듈 명세서

① ILP 스케줄러 (PuLP)

- 목적: 사용자 일정·제약조건 기반 청소 스케줄 초안 생성
- 입력: 사용자 일정, 청소작업 목록
- 출력: 1주일 청소 일정(요일, 시간, 작업) 리스트
- 주요 기능: 목적 함수 최대화, 제약조건 검증
- 상세 요구사항: 조건을 최대한으로 만족하는 최적의 시간표를 수학적 기법으로 구한다.
 - 1주일 청소 태스크의 개수 제한
 - 1주일 청소 태스크의 시간의 합 제한
 - 구글 캘린더 일정과 수면시간을 제외한 시간에 청소 배정
 - 하나의 청소 태스크를 1주일에 여러번 배정할 때 권장 주기를 준수
 - 미룬지 오래됐거나 권장 주기가 짧은 작업을 우선 배정

② RL 엔진 (Tianshou PPO)

- 목적: 개인화·업그레이드된 사용자 맞춤형 스케줄 산출
- 입력: ILP 스케줄, 시간대 별 청소 확률 데이터, 온보딩 데이터, 지난주 청소 피드백 데이터
- 출력: 개인화된 청소 일정(요일, 시간, 작업) 리스트
- 주요 기능: PPO 알고리즘으로 개인화된 최적 스케줄 생성
- 상세 요구사항: 과거 청소 기록, 온보딩, 피드백 데이터를 기반으로 보상 함수 학습.
 - 구글 캘린더 일정과 수면시간을 제외한 시간에 청소 배정
 - 같은 요일 및 시간에 청소 태스크를 2개 이상 배정 시 패널티를 부여해
 - 늦은 밤이나 새벽 시간대 배정 시 패널티를 부여해 적절한 시간에 일정이 배정
 - 시급한(미룬지 오래된) 청소 작업에는 높은 보상을 부여해 우선적으로 처리
 - 과거 청소 성공 이력이 많은 시간대와, 사전에 사용자가 입력한 선호 시간대에 일정이 배정되도록 유도
 - 사용자의 청소 스타일(한번에 몰아서or조금씩 나눠서)에 따라 슬롯 중복 패널티를 다르게 부여해 각 청소 태스크가 분산 배정되는 정도를 조절

③ 날씨 모듈 (OpenWeatherMap API)

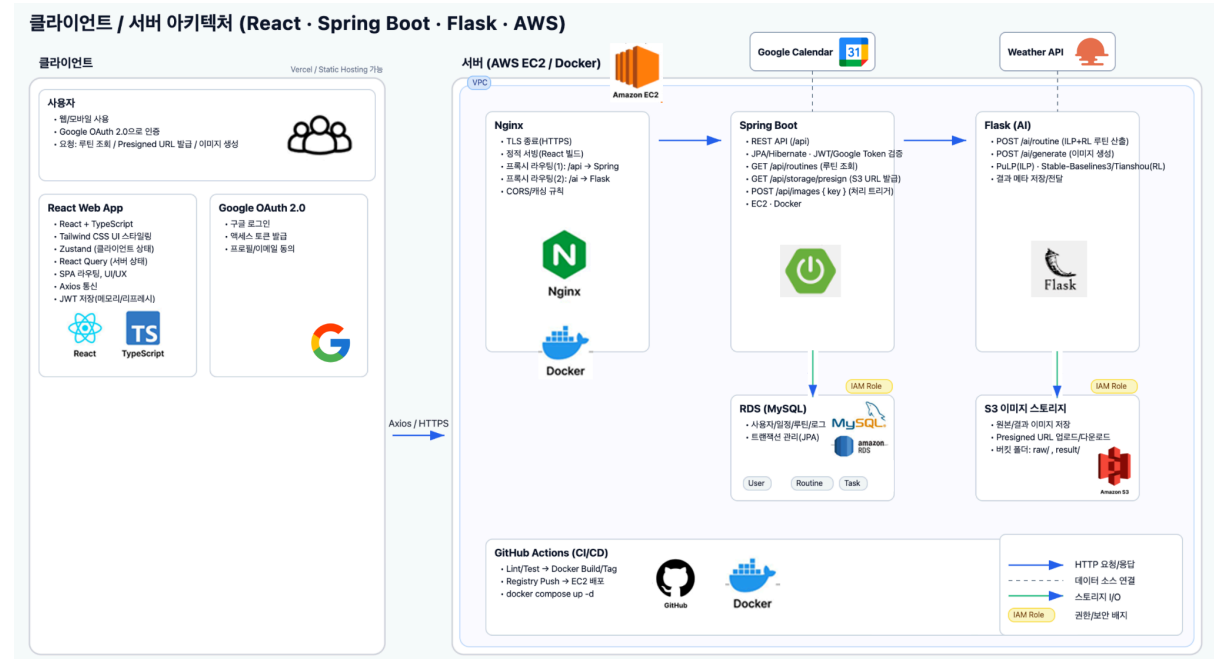
- 목적: 실시간 날씨 데이터를 활용한 청소 작업 추천
- 입력: 사용자 위치 정보, OpenWeatherMap API 날씨 데이터(비·미세먼지·습도 등)
- 출력: 날씨 상황에 맞는 청소 작업
- 주요 기능:
 - 지역 날씨 실시간 수집

- 날씨 조건에 따라 청소 작업 자동 추천

④ 이미지 모듈 (Grounded-Segment-Anything + Smart-Eraser)

- 목적: 청소 동기부여를 위한 시각적 비교 제공
- 입력: 사용자가 업로드한 방 사진
- 출력: 불필요한 객체가 제거된 가상의 청소 후 이미지
- 주요 기능:
 - SAM/Grounded-Segment-Anything으로 객체 세그멘테이션·마스킹
 - Smart-Eraser로 불필요한 물체 자연스럽게 제거 및 배경 채움

3.2 전체 시스템 구성



본 서비스는 사용자-클라이언트-게이트웨이-백엔드-AI 서비스-데이터 저장소로 이어지는 계층형 아키텍처이다. 사용자 입력(일정·사진·피드백)은 표준 **HTTPS**를 통해 수집되고, 스케줄 추천·이미지 생성·파일 저장이 분리된 모듈에서 처리된다.

3.2.1 구성 요소와 역할

1) 사용자 & 클라이언트(웹)

- 사용자: 웹/모바일 브라우저를 통해 서비스에 접속한다.
- React Web App**: React + TypeScript 기반 SPA이다. Tailwind CSS로 UI를 구성하고, Zustand로 클라이언트 상태를, React Query로 서버 상태를 캐싱한다. Axios/HTTPS로 서버 API를 호출한다. JWT는 메모리(필요시 리프레시 토큰 포함)에 보관한다.
- 인증: **Google OAuth 2.0**으로 로그인하며, 액세스 토큰과 프로필/이메일 동의를 획득한다.

2) 엣지/게이트웨이

- Nginx**: TLS 종료와 정적 파일 서빙을 담당한다. 리버스 프록시로 라우팅 규칙을 적용하여 /api는 백엔드(Spring), /ai는 AI(Flask)로 분기한다. CORS/캐싱 정책을 관리한다.

3) 백엔드 API

- Spring Boot**: 도메인 API를 제공한다.
 - 인증/권한: **JWT** 검증(필요 시 Google 토큰 검증)
 - 주요 엔드포인트:
 - GET /api/routines** — 맞춤 루틴 조회
 - GET /api/storage/presign** — **S3 Presigned URL** 발급
 - POST /api/images {key}** — 업로드 완료 후 처리 트리거
 - 데이터 접근: **JPA/Hibernate**로 RDS(MySQL) 연동
 - 배포: **AWS EC2** 컨테이너(Docker)로 운영

4) AI 서비스

- **Flask (Python):** 스케줄·이미지 관련 연산을 담당하는 마이크로서비스이다.
 - **POST /ai/routine** — ILP + RL 기반 루틴 산출
 - **POST /ai/generate** — 청소 후 예상 이미지 생성(시각적 동기부여)
- 알고리즘/라이브러리
 - **PuLP:** 정수선형계획(ILP)으로 초기 스케줄 최적화
 - **Tianshou:** 강화학습으로 루틴 적응형 개선

5) 데이터 저장소

- **RDS (MySQL):** 사용자, 일정, 루틴, 태스크, 피드백, 로그를 저장한다. 트랜잭션은 JPA로 관리한다.
- **S3:** 이미지 원본/결과 파일을 저장한다. 클라이언트는 백엔드가 발급한 **Presigned URL**로 직접 업로드/다운로드한다(서버는 메타만 관리).
- **IAM Role:** EC2 인스턴스의 S3/RDS 접근을 역할 기반 권한으로 제한한다.

6) 외부 연동

- **Google Calendar API:** 사용자 일정 동기화(초기/주기적).
- **OpenWeatherMap API:** 강수·미세먼지·습도 등 환경 데이터를 수집하여 루틴/권고안에 반영한다.

7) 배포/운영

- **Docker:** Nginx, Spring, Flask를 컨테이너로 패키징한다.
- **GitHub Actions(CI/CD):** Lint/Test → Docker Build/Tag → Registry Push → EC2 배포 → **docker compose up -d** 파이프라인으로 자동 배포한다.
- **VPC:** EC2(퍼블릭 서브넷), RDS(프라이빗 서브넷)로 분리하여 보안을 강화한다.

3.2.2 데이터 흐름(요약)

1. 사용자가 **Google OAuth 2.0**으로 로그인한다(JWT 발급).
2. 클라이언트가 **GET /api/routines**로 개인화 루틴을 조회한다.
3. 이미지 업로드가 필요하면 **GET /api/storage/presign**으로 **Presigned URL**을 받고, 클라이언트가 **S3**로 직접 업로드한다.
4. 업로드 완료 후 **POST /api/images {key}**로 백엔드를 통지하면, 백엔드는 필요 시 **Flask**에 **/ai/generate** 또는 **/ai/routine**을 호출한다.
5. Flask는 **PuLP/Tianshou**로 계산·생성 작업을 수행하고, 결과 메타를 백엔드에 전달한다.
6. 백엔드는 **RDS**에 상태를 저장하고 응답을 반환한다. 클라이언트는 결과 이미지를 **S3**에서 읽고, 화면에 표시한다.

3.2.3 사용 오픈소스/외부 모듈

- 프론트엔드: React, TypeScript, Tailwind CSS, Zustand, React Query, Axios
- 백엔드: Spring Boot, JPA/Hibernate, JWT
- AI/알고리즘: Flask, **PuLP**, **Tianshou**, **Grounded-Segment-Anything**, **Smart-Eraser**
- 엣지/런타임: Nginx, Docker
- 클라우드: AWS EC2, RDS(MySQL), S3, IAM, VPC
- 외부 API: **Google OAuth 2.0** / **Calendar API**, **Weather API**(OpenWeatherMap/기상청 등)
- **CI/CD:** GitHub Actions